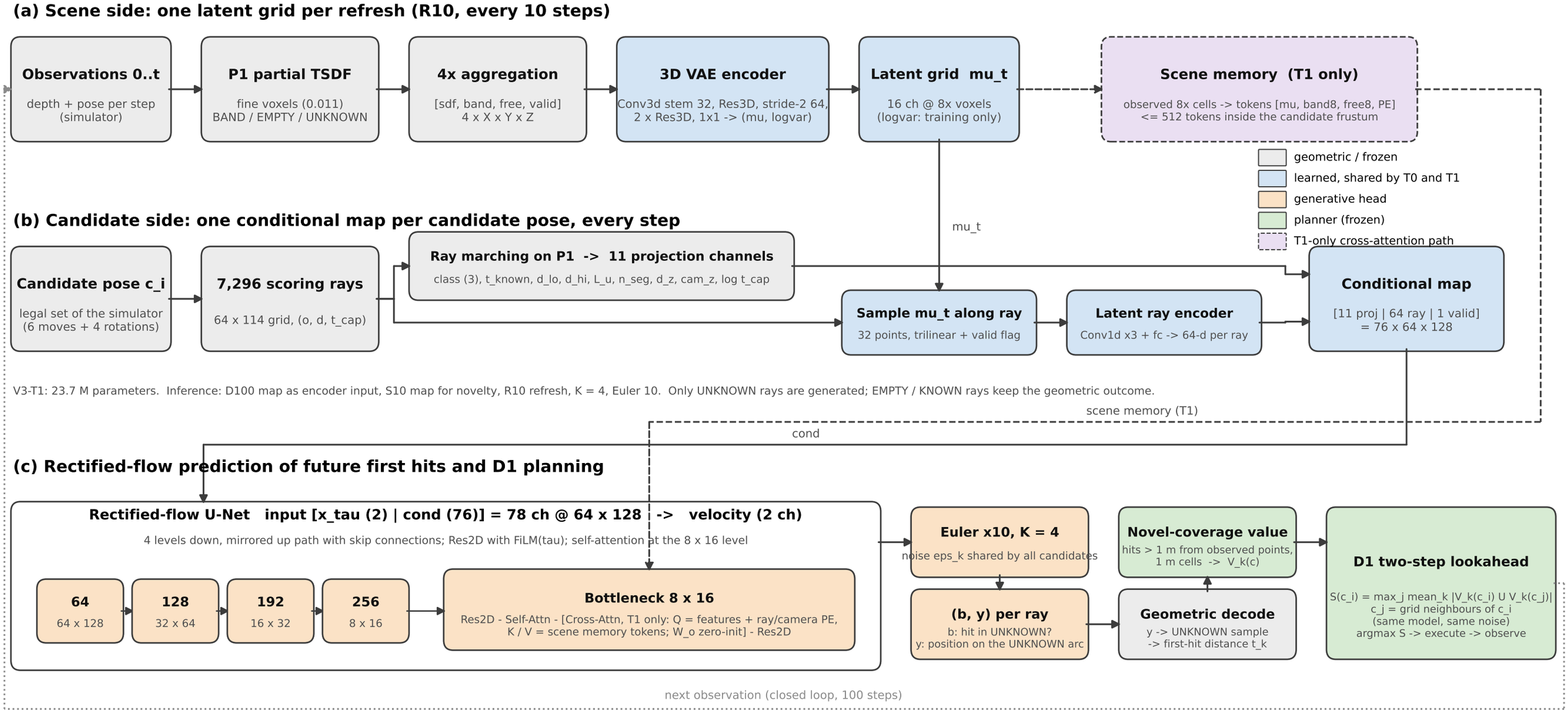
(a) Scene side: one latent grid per refresh (R10, every 10 steps)
Observations 0..t
depth + pose per step (simulator)
P1 partial TSDF
fine voxels (0.011) BAND / EMPTY / UNKNOWN
4x aggregation
[sdf, band, free, valid] 4 x X x Y x Z
3D VAE encoder
Conv3d stem 32, Res3D, stride-2 64, 2 x Res3D, 1x1 -> (mu, logvar)
Latent grid mu_t
16 ch @ 8x voxels (logvar: training only)
Scene memory (T1 only)
observed 8x cells -> tokens [mu, band8, free8, PE] <= 512 tokens inside the candidate frustum
geometric / frozen
learned, shared by T0 and T1
generative head
planner (frozen)
T1-only cross-attention path
(b) Candidate side: one conditional map per candidate pose, every step
mu_t
Candidate pose c_i
legal set of the simulator (6 moves + 4 rotations)
7,296 scoring rays
64 x 114 grid, (o, d, t_cap)
Ray marching on P1 -> 11 projection channels
class (3), t_known, d_lo, d_hi, L_u, n_seg, d_z, cam_z, log t_cap
Sample mu_t along ray
32 points, trilinear + valid flag
Latent ray encoder
Conv1d x3 + fc -> 64-d per ray
Conditional map
[11 proj | 64 ray | 1 valid] = 76 x 64 x 128
V3-T1: 23.7 M parameters. Inference: D100 map as encoder input, S10 map for novelty, R10 refresh, K = 4, Euler 10. Only UNKNOWN rays are generated; EMPTY / KNOWN rays keep the geometric outcome.
scene memory (T1)
cond
(c) Rectified-flow prediction of future first hits and D1 planning
Rectified-flow U-Net input [x_tau (2) | cond (76)] = 78 ch @ 64 x 128 -> velocity (2 ch)
4 levels down, mirrored up path with skip connections; Res2D with FiLM(tau); self-attention at the 8 x 16 level
64
64 x 128
128
32 x 64
192
16 x 32
256
8 x 16
Bottleneck 8 x 16
Res2D - Self-Attn - [Cross-Attn, T1 only: Q = features + ray/camera PE, K / V = scene memory tokens; W_o zero-init] - Res2D
Euler x10, K = 4
noise eps_k shared by all candidates
(b, y) per ray
b: hit in UNKNOWN?
y: position on the UNKNOWN arc
Geometric decode
y -> UNKNOWN sample -> first-hit distance t_k
Novel-coverage value
hits > 1 m from observed points, 1 m cells -> V_k(c)
D1 two-step lookahead
S(c_i) = max_j mean_k |V_k(c_i) U V_k(c_j)|
c_j = grid neighbours of c_i
(same model, same noise)
argmax S -> execute -> observe
next observation (closed loop, 100 steps)